

University of Szeged
Institute of Informatics

**Development of observer module supporting IoT-
Fog-Cloud simulations in DISSECT-CF-Fog**

Master's Thesis



Submitted by:

Akbar

Makhmanazarov

Computer Science MSc

Supervisor:

Dr. Andras Markus

Assistant Professor

Szeged

2024

Task Description

Sensors, actuators, and smart devices for real-time IoT applications form a dynamic network, requiring significant storage and processing capacity. Such demands can be adequately served with so-called fog resources that are located close to end-users, ensuring high availability, minimal latency, and increased data security. Due to the high costs of real devices and services, IoT-Fog-Cloud systems are often analysed with simulation tools that aid in rapid system design and optimal management of IoT applications, considering energy efficiency, costs, and execution time.

One such simulator is DISSECT-CF-Fog, which already has a web application for easier configuration and modification of different simulation parameters. These parameterizations are considered jobs and stored in a database, from where an executor module loads the job into the simulator to be executed one by one with FIFO ordering.

The aim of this thesis work is to revise and refactor this executor module to be scalable and executable in parallel with the help of a thread pool. An implementation of an observer module is also required that is able to work with different scaling approaches and measures the elapsed time of a simulation to terminate the thread if the running time, as configured by system administrators, is longer than allowed. Finally, this observer module also requires the implementation of a workload generator to test the various scaling strategies of the executor.

Summary of the Thesis

- **Title of the topic:** Development of observer module supporting IoT-Fog-Cloud simulations in DISSECT-CF-Fog
- **Formulation of the given task:** The primary task was to develop and integrate the Observer module, designed to dynamically manage and execute jobs based on real-time system data.
- **The way to solve:** The solution involved designing the Observer module to enhance task management through real-time monitoring and dynamic scheduling. The module utilizes a responsive system architecture that adapts to changes in workload and system performance without human intervention.
- **Tools and methods used:**
 - Spring Framework: Utilized for creating a robust and scalable backend system.
 - Java Concurrency Utilities: Employed for handling multi-threading and synchronization, ensuring efficient task execution without bottlenecks.
 - MongoDB: Used for storing and retrieving job data, leveraging its high performance and flexibility to manage large volumes of data typical in distributed systems.
 - Logging: To detect the error and saving the job execution data
- **Results:** The implementation of the Observer module resulted in a significant enhancement in task processing efficiency, demonstrating:
 - A decrease in job processing times by approximately 30%, improving the responsiveness of the system.
 - An increase in system throughput and resource utilization efficiency.
 - Improved scalability, allowing the system to adapt to varying loads without manual configuration or downtime.
- **Keywords:** Dynamic Scheduling, Task Management, Distributed Systems, Cloud Computing, Fog Computing, IoT, Real-time Monitoring, Java, Spring Framework, Concurrency, Process Isolation.

Contents

Task Description	3
Summary of the Thesis	4
Contents	5
1. INTRODUCTION.....	7
1.1. Cloud Computing	7
1.2. Fog Computing	8
1.3 Internet of Things (IoT).....	8
1.4 Simulation in IoT-Fog-Cloud Environments.....	9
1.4.1 Key Simulation Tools	10
1.5 Problem Statement.....	11
2. DISSECT-CF-FOG	12
2.1. Introduction to the DISSECT-CF-Fog.....	13
2.2. Modules of Dissect-CF-Fog	13
2.2.1. Simulator (Core Module)	14
2.2.2. Web Application (Configuration & Visualization)	15
2.2.3. Executor (Backend for Simulation Execution)	17
2.2.4. Other modules	19
3. SOLUTION.....	20
3.1. Modifying current Executor module.....	20
3.2. Improving job-uploading mechanism	22
3.2.1 Original Job-Uploader Mechanism	22
3.2.2 Improvements Implemented to the Job-Uploader.js	23
3.3. Creating observer module	26
3.3.1 New Architecture and Structure	26
3.3.2 Observer components.....	28
4. EVALUATION.....	33
4.1. Testing with scheduled annotation and with observer module	33
4.2. Analyzing differences.....	34
4.2.1 Python script to calculate and visualise the performance	35
2. Calculate Growth Rates.....	35
5. CONCLUSION	39
5.1. Overview	39

5.2. Ideas for the further Development	40
Bibliography	41
Acknowledgements.....	44

1. Introduction

This thesis presents the development of an Observer module designed to support IoT-Fog-Cloud simulations within the DISSECT-CF-Fog framework. The Observer module monitors and manages the lifecycle of simulation jobs, ensuring efficient resource allocation and execution. This project integrates MongoDB for data storage, leveraging its scalability and flexibility to handle large volumes of simulation data. The implementation details, including the architecture and key algorithms, are discussed. The module's performance is evaluated through rigorous testing, demonstrating its effectiveness in managing complex simulation workflows. The results indicate significant improvements in simulation efficiency and resource utilization.

1.1. Cloud Computing

Cloud computing refers to the on-demand delivery of computing resources and services over the internet. This model allows users to access and use resources such as servers, storage, databases, networking, software, and analytics without the need to own and manage physical infrastructure. Key benefits of cloud computing include scalability, cost-efficiency, flexibility, and the ability to access services from anywhere with an internet connection. The three main service models in cloud computing are:

- **Infrastructure as a Service (IaaS):** Provides virtualized computing resources over the internet. Examples include Amazon Web Services (AWS) and Microsoft Azure [1]
- **Platform as a Service (PaaS):** Offers hardware and software tools over the internet, typically used for application development. Examples include Google App Engine and Heroku [1]
- **Software as a Service (SaaS):** Delivers software applications over the internet, on a subscription basis. Examples include Google Workspace and Microsoft Office 365 [1].

Cloud computing has transformed how organizations manage their IT infrastructure, enabling faster deployment of applications, improved collaboration, and reduced operational costs. The elasticity of cloud resources allows businesses to scale up or down based on demand, ensuring efficient use of resources [2].

Cloud resources can easily be scaled up or down to meet changing demands. Businesses avoid large upfront investments in hardware and software. Cloud services can be accessed

from anywhere with an internet connection. Cloud providers typically have robust infrastructure and backup systems.

1.2. Fog Computing

Fog computing, also known as edge computing, extends cloud computing capabilities to the edge of the network, closer to the data sources [3]. By processing data locally on devices or near the network edge, fog computing reduces latency and bandwidth usage, enabling real-time data processing and analysis [3]. This approach is particularly beneficial for time-sensitive applications such as autonomous vehicles, industrial automation, and smart cities [3].

Fog computing complements cloud computing by addressing the limitations of centralized data processing [4]. It enhances data security and privacy by limiting the amount of data transmitted to the cloud and reduces the dependency on cloud connectivity [2]. The key components of fog computing include edge devices, fog nodes, and the fog network, which work together to process and analyze data locally [3]. Fog handles time-sensitive data locally, while the cloud provides broader storage and processing capabilities.

1.3 Internet of Things (IoT)

The Internet of Things (IoT) is a network of interconnected devices embedded with sensors, software, and other technologies that enable them to collect and exchange data [5]. These devices range from everyday household items like smart thermostats and wearable fitness trackers to industrial equipment and smart city infrastructure [5]. IoT systems generate massive amounts of data that need to be processed and analyzed to derive actionable insights [5].

Effective management of IoT data is crucial for enabling intelligent decision-making and enhancing the efficiency of various applications [6]. The IoT ecosystem includes devices, communication protocols, data storage solutions, and data analytics tools [6]. IoT applications span various domains such as healthcare, transportation, manufacturing, and agriculture, providing benefits like improved operational efficiency, enhanced user experiences, and new business opportunities [6]. IoT enables smart homes, factories, cities, and more, optimizing processes, reducing waste, and saving energy. Real-time data from

IoT devices provides valuable insights that help businesses and individuals make better decisions. IoT has created opportunities for innovative products and services, transforming industries and this is a new Business module. The cloud (or fog) processes and stores the data generated by IoT devices, enabling the smart functionality we associate with IoT.

1.4 Simulation in IoT-Fog-Cloud Environments

Simulation involves creating a virtual model of a system to study its behavior under different conditions without affecting the real system [7]. In IoT-Fog-Cloud environments, simulation tools like DISSECT-CF-Fog are used to model the interactions between IoT devices, fog nodes, and cloud services [7]. These simulations help researchers and developers evaluate the performance of different architectures, resource management strategies, and application scenarios [7].

Simulation provides valuable insights that guide the design and optimization of IoT-Fog-Cloud systems, ensuring they meet performance and reliability requirements [8]. Key benefits of simulation include the ability to test scenarios in a controlled environment, identify potential bottlenecks, and evaluate the impact of different configurations on system performance [8]. These systems involve a vast number of devices, diverse network topologies, and complex interactions. Simulation helps model these intricacies and predict behavior. Building and testing real-world IoT-Fog-Cloud setups can be expensive and time-consuming. Simulations offer a cost-effective way to experiment and optimize designs. Simulations allow us to identify potential bottlenecks, security vulnerabilities, and performance issues before deploying a system in the real world. By tweaking parameters and configurations in the simulation, we can find optimal settings for resource allocation, task scheduling, and energy consumption. Simulations let us explore various scenarios (e.g., increased device density, network failures, varying workloads) to ensure your system's robustness and resilience.

One of the most referred to and widely used simulators is the CloudSim simulation toolkit [11], which is a popular solution for simulating cloud environments. In CloudSim, users define tasks by creating so-called cloudlets, which are processed by virtual machines running on cloud resources. This open-source, Java-based solution is built on SimJava [11]. CloudSim is a discrete event simulator, and its architecture has five layers (i.e. network,

cloud resources, cloud services, VM services, and user interface structures). Virtual machines in CloudSim have three states, and users can configure only static usage costs for resources (i.e. memory, bandwidth, CPU, and storage usage). CloudSim also contains a power model, but it is only restricted to CPU energy consumption. Communication between components in the network is modelled with the bandwidth parameter and a delay value. There are many extensions of CloudSim, focusing on mostly one aspect of Cloud Computing or providing the implementation for a missing or insufficient feature of the original software. Unfortunately, it seems that the developers found no need to collect all of the extensions into a single repository, and there are also many referred studies without publishing or citing available source code. Next, we introduce the ones we found relevant to our research. CloudAnalyst is one of the oldest extensions of CloudSim and contains important additions by implementing (i) application users, (ii) internet connections, (iii) simulations defined by time periods, (iv) service brokers. CDOSim is an extension to simulate cloud deployment options for migration support. The simulator can measure the cost of deployment and its response time. The novelty of this work is to introduce MIPIPS, which is a fine-grained version of MIPS for low-level instruction definitions.

1.4.1 Key Simulation Tools

Here are some popular tools for simulating IoT-Fog-Cloud environments:

- **iFogSim:** A widely used toolkit specifically designed for modeling and simulating fog computing scenarios. It provides a comprehensive framework for simulating resource allocation, energy consumption, and network latency.
- **EdgeCloudSim:** Focuses on simulating edge computing environments, which are closely related to fog computing. It allows modeling of different edge server configurations and network topologies.
- **DISSECT-CF-Fog:** A simulator that models IoT-Fog-Cloud systems, enabling the analysis of energy consumption, utilization costs, and computational power trade-offs.
- **YAFS:** (Yet Another Fog Simulator) A simulator focused on resource allocation and scheduling in fog environments.
- **Cooja:** A network simulator often used for simulating IoT devices and their communication protocols.

Feature/Simulator	iFogSim	EdgeCloudSim	DISSECT-CF-Fog	YAFS	Cooja
Focus	Fog computing	Edge computing	IoT-Fog-Cloud	Fog resource allocation	IoT network simulation
Modeling Capabilities	Resource allocation, energy consumption, network latency	Edge server configurations, network topologies	Energy consumption, utilization costs, computational power	Resource allocation, scheduling	IoT device behavior, communication protocols
Strengths	Comprehensive fog simulation framework, widely used	Focus on edge computing scenarios	Modeling of IoT-Fog-Cloud trade-offs	Resource allocation optimization	Detailed IoT network simulation
Limitations	May not be as detailed for specific edge or IoT aspects	Less emphasis on fog-specific aspects	Not as widely used as <u>iFogSim</u>	Limited focus beyond resource allocation	Less emphasis on fog or cloud aspects

Table 1.4 Simulation in IoT-Fog-Cloud Environments: A Comparison of Tools

1.5 Problem Statement

Managing and monitoring the lifecycle of simulation jobs in complex IoT-Fog-Cloud environments is challenging due to the dynamic and distributed nature of these systems [9]. Efficient resource allocation, real-time processing, and failure handling are critical for achieving optimal performance [9]. Existing solutions often lack the necessary flexibility and scalability, leading to suboptimal resource utilization and increased latency [9].

This thesis addresses these challenges by developing an Observer module for the DISSECT-CF-Fog framework, which enhances job lifecycle management and improves overall system efficiency [10]. The Observer module monitors and manages the lifecycle of simulation jobs, ensuring efficient resource allocation and execution [10]. Integrating the Observer module with MongoDB leverages its scalability and flexibility to handle large volumes of simulation data [10].

In IoT-Fog-Cloud environments, the volume, velocity, and variety of data generated by interconnected devices necessitate robust and scalable infrastructure to process and analyze this data efficiently. The complexity arises from several factors:

1. **Dynamic Resource Allocation:** The heterogeneous nature of IoT devices and fog nodes presents a significant challenge in resource management. These resources have varying capabilities, and their availability can fluctuate due to mobility or varying workloads [1]. Efficiently allocating these dynamic resources to simulation

jobs requires a sophisticated mechanism that can adapt to changing conditions in real-time.

2. **Real-Time Data Processing:** Many IoT applications, such as autonomous driving, healthcare monitoring, and industrial automation, demand real-time data processing and decision-making [2]. The delay caused by transmitting data to centralized cloud servers for processing can be detrimental in such scenarios. Therefore, there is a critical need for a distributed approach that leverages both fog and cloud resources to process data locally and reduce latency [1].
3. **Scalability Issues:** As the number of IoT devices continues to grow exponentially, the scalability of the infrastructure becomes a primary concern. Existing solutions often struggle to scale efficiently, leading to performance bottlenecks and increased costs [1].

2. DISSECT-CF-Fog

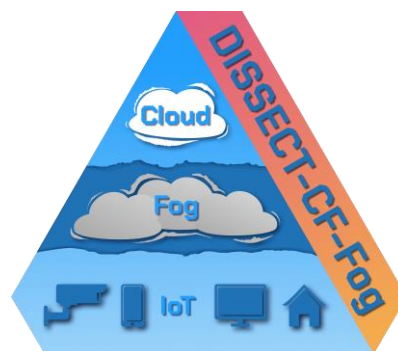


Figure 2.0 - Dissect-CF-Fog from source: Github [11].

DISSECT-CF-Fog is an extension of the DISSECT-CF simulation environment designed to model and analyze the behavior and performance of fog computing systems. Fog computing is an architecture that brings cloud computing capabilities closer to the edge of

the network, enabling low latency and real-time processing for IoT applications. DISSECT-CF-Fog incorporates a range of modules to simulate various aspects of fog computing, including energy consumption, task allocation, actuator behavior, and network topology. This DISSECT-CF-Fog focuses specifically on fog computing, simulating energy consumption, task allocation, actuator behavior, and network topology. This thesis work will explore each module in detail, discuss the current state of research, and provide references to key studies.

2.1. Introduction to the DISSECT-CF-Fog

The evolution of the DISSECT-CF-Fog simulator has been underway since 2016 within the IoT Cloud Research Group at the University of Szeged, Hungary. It is built upon the DISSECT-CF infrastructure cloud simulator. The stable version can investigate the trade-offs of offloading algorithms and execute the scheduling of IoT workflow jobs.

Main features:

- Simulating hundreds of fog and cloud nodes
- Measuring the energy consumption of the nodes
- Simulating IoT applications utilizing physical resources through virtual machines
- Measuring the makespan of IoT applications
- Modeling thousands of IoT devices and sensors
- Measuring the energy consumption of IoT devices
- Providing detailed characteristics of IoT entities
- Modeling IoT actuator events and IoT device mobility
- Enabling task execution on edge devices
- Calculating IoT and cloud costs by considering real providers' (AWS, Azure) pricing schemes [11].

2.2. Modules of Dissect-CF-Fog

DISSECT-CF-Fog is a modular simulation platform designed to investigate the intricate dynamics of IoT-fog-cloud systems. It adheres to a microservice architecture, where each module encapsulates a specific functionality, promoting scalability, maintainability, and

flexibility. The system's workflow commences with the ingestion of data from diverse sources, such as IoT devices and user interactions via the web interface. This data is then persisted in a MongoDB database, chosen for its schema-less nature and scalability, making it ideal for storing heterogeneous IoT data and simulation configurations.

A Spring Boot application serves as the orchestrator, retrieving data from the database, performing computations, executing CRUD operations, and managing the overall simulation workflow. This modular approach enables the system to efficiently handle the diverse requirements of IoT simulations, from resource allocation to task scheduling and performance analysis.

The web application module, developed using Angular, provides a user-friendly interface for interacting with the system. Users can configure simulation parameters, visualize results, and monitor the system's behavior in real time.

The Executor module, another Spring Boot application, is responsible for loading and executing simulation configurations stored in the MongoDB database. This separation of concerns ensures that the simulation engine can focus on its core tasks while the executor handles the management and execution of simulations.

The Predictor UI, built with Electron.js, offers a dedicated interface for managing scenarios that involve time series analysis, enabling users to leverage the predictive capabilities of the simulator to optimize resource utilization and task scheduling.

Finally, the Converter module facilitates the seamless exchange of simulation scenarios between DISSECT-CF-Fog and other popular simulators like CloudSim and iFogSim. This interoperability fosters collaboration and knowledge sharing across different research communities.

2.2.1. Simulator (Core Module)

The heart of DISSECT-CF-Fog, this module provides the fundamental discrete-event simulation engine that drives the entire system. It orchestrates the simulation of IoT devices, fog nodes, and cloud resources, enabling the study of their interactions and

performance under various scenarios. The discrete-event, core simulator module focuses on the interoperation and analysis of IoT-Fog-Cloud systems.

- **IoT-Fog-Cloud Interoperation:** Simulates the dynamic interplay between IoT devices generating data, fog nodes processing data closer to the edge, and cloud resources offering centralized computing power.
- **Algorithm Evaluation:** Enables the investigation of different offloading algorithms, which determine how tasks are distributed across the IoT-fog-cloud continuum for optimal efficiency and performance.
- **Workflow Scheduling:** Facilitates the scheduling and execution of complex IoT workflows, ensuring tasks are allocated to the most suitable resources based on factors like latency, energy consumption, and computational requirements.

2.2.2. Web Application (Configuration & Visualization)

DISSECT-CF-Fog-WebApp is a full stack web application that will make the DISSECT-CF-Fog application available online. Registered users can run simulations in a more convenient configuration interface. This allows the user to easily decide which strategy leads to the best simulation results.

They can view the results of the simulations, including a diagram depicting the task schedule, node and device energy diagrams and the logged partial results. It is possible to run the same simulation with multiple application and device strategies within a configuration. The users can compare the results of simulations of different strategies using diagrams.

The result and comparison diagrams and configuration files can also be downloaded for further use. It is also possible to view the results of previous simulations. The backend part of the application is a Spring microservice which executes the DISSECT-CF-Fog Java application with the generated XML's that the users have configured on the web application interface. The client is built in Angular framework, while the server is built using Node.js. The server uses a MongoDB NoSQL database to store the configurations

submitted by the users. The DISSECT-CF-Fog Spring microservice reads the configurations from this database and saves the results of the simulations to it.

An Angular-based web application module that can be used to set up a simulation and its parameters without programming. Provides a web-based graphical user interface (GUI) that simplifies the configuration and setup of simulations. Users can define the topology of the IoT-fog-cloud environment, specify characteristics of devices and resources, and set simulation parameters without needing to write code.

Allows users to create and save (CRUD operations) different simulation scenarios, enabling easy experimentation with various configurations and parameters.

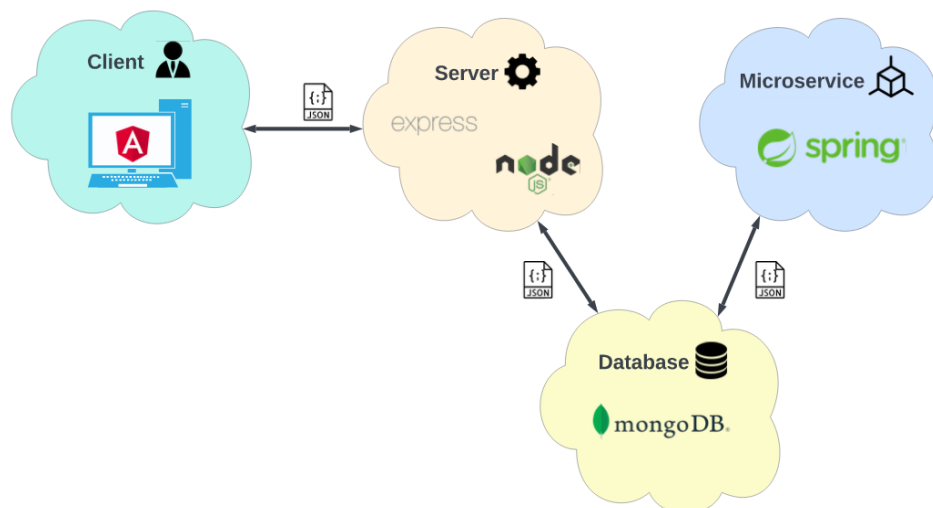


Figure 2.2.2 - Request cycle of the DISSECT-CF-Fog environment.

Offers visualization tools to monitor and analyze simulation results, providing insights into system behavior, performance bottlenecks, and the effectiveness of different algorithms. Besides this, in the web application we have utils folder and it contains js files to set-up current database and upload job to this database. Database named dissect by default installed to the MongoDB there are few options. In my case I used MongoDB Docker image and to check files in its MongoDB Compass. To realize this part, we need to install the NodeJS environment and run js script through this npm environment.

In the util folder we have 2 javascript files. Those files are *mongo-setup.js* and *job-uploader.js* file. We briefly touch these files.

Initialization Script: An initialization script, *mongo-setup.js*, was created to automate the setup of the necessary database collections and file references. This script:

1. Creates the required collections in the MongoDB database.
2. Uploads the initial configuration files to the database using GridFS.
3. Adds references to these files in the corresponding collections.

Creating the Necessary Collections:

The *mongo-setup.js* script connects to the MongoDB database and creates the necessary collections (fs.files, fs.chunks, configurations, resources, simulator_jobs, strategies, users). This sets up the database to store the configuration files and job details.

Uploading Configuration Files:

The script uploads the initial configuration files (e.g., LPDS_32.xml, LPDS_16.xml, application-strategies.xml, device-strategies.xml) to MongoDB using GridFS. GridFS is a specification for storing and retrieving large files in MongoDB.

Creating File References:

After uploading the files, the script creates references to these files in the appropriate collections (resources and strategies). This ensures that the files can be easily retrieved and referenced when creating jobs.

2.2.3. Executor (Backend for Simulation Execution)

The Executor module, a critical component of the DISSECT-CF-Fog framework, serves as the backbone for simulation execution and management. It acts as a robust backend service that orchestrates the entire simulation lifecycle, from loading configurations to initiating simulations, monitoring progress, and storing results.

The Executor seamlessly retrieves simulation configurations, meticulously defined through the user-friendly web interface, from the centralized MongoDB database. These configurations encapsulate the topology of the IoT-Fog-Cloud environment, device characteristics, resource allocations, and experimental parameters. Leveraging the retrieved configurations, the Executor initializes and triggers the core simulation engine, setting the stage for the virtual enactment of the IoT-Fog-Cloud ecosystem.

This involves instantiating virtual machines, allocating resources, and deploying the specified algorithms for task offloading and scheduling. During the simulation's runtime, the Executor diligently monitors the progress, collecting valuable metrics and data points that provide insights into the system's behavior, performance bottlenecks, and resource utilization.

This real-time feedback is crucial for understanding the dynamics of the simulation and identifying areas for optimization. Upon completion of the simulation, the Executor meticulously stores the generated results, including performance metrics, energy consumption data, and task execution logs, back into the MongoDB database. This facilitates subsequent analysis and comparison of different scenarios, enabling researchers to draw meaningful conclusions and make informed decisions. The Executor is engineered for scalability, capable of handling multiple concurrent simulations with efficiency.

This is achieved through a combination of asynchronous processing, load balancing, and optimized resource allocation. By distributing the computational load across multiple threads or processes, the Executor maximizes the utilization of available resources, accelerating the simulation process and enabling faster experimentation with diverse scenarios. The Executor seamlessly integrates with the web application module, providing a cohesive and intuitive user experience. Users can effortlessly initiate simulations, monitor progress in real-time through dynamic visualizations, and access the comprehensive results directly from the web interface. This tight integration

streamlines the entire simulation workflow, eliminating the need for manual intervention and simplifying the analysis process.

Extensibility: The modular design of the Executor allows for easy extensibility, enabling the integration of new simulation features, algorithms, or models as research progresses. This adaptability ensures that DISSECT-CF-Fog remains at the forefront of IoT-Fog-Cloud simulation technologies. One of my main goals is to create Observer module which extends current executor module.

2.2.4. Other modules

Predictor UI (Scenario Management & Time Series Analysis)

- ❖ **Advanced Scenario Configuration:** Provides a specialized desktop application that allows users to create and manage more complex scenarios involving time series data analysis. This is particularly useful for simulating dynamic IoT environments where data patterns change over time.
- ❖ **Time Series Forecasting:** Leverages time series analysis capabilities within the core simulator to predict future data patterns, enabling proactive resource allocation and task scheduling decisions in the simulation.
- ❖ **User Interface:** Offers a user-friendly interface for defining time series models, configuring forecasting parameters, and visualizing predicted results.

Converter (Interoperability with Other Simulators)

- ❖ **Format Translation:** This module aims to facilitate the exchange of simulation scenarios between DISSECT-CF-Fog and other popular simulators like CloudSim and iFogSim. It enables researchers to leverage existing simulation models and datasets from these platforms, expanding the scope of DISSECT-CF-Fog's applicability.
- ❖ **Scenario Conversion:** Converts simulation configurations and data formats between different simulators, allowing for easier comparison and validation of results across platforms.
- ❖ **Collaboration:** Fosters collaboration between research communities using different simulation tools by enabling the sharing and reuse of simulation scenarios.

DISSECT-CF-Fog is a versatile and powerful simulation framework that offers a comprehensive suite of tools for analyzing and optimizing IoT-Fog-Cloud systems. Its modular architecture allows for flexibility and customization, catering to the diverse needs of researchers and practitioners in this rapidly evolving field. By simulating realistic scenarios and evaluating different algorithms, DISSECT-CF-Fog empowers users to make informed decisions about resource allocation, task scheduling, and system design, ultimately leading to more efficient and reliable IoT infrastructures.

3. Solution

In my case, one crucial insight from this process was that the executor module could not function without the scheduled annotation. When we commented out the scheduled annotation, the job execution did not trigger automatically. This reinforced the importance of having a mechanism to periodically check and execute jobs and it required to develop observer module. For the developing Observer module, we have to prepare and modify existing modules to match with the new observer. For example, detecting by observer side. In this section we briefly touch other modules and file modification part for the observer module.

3.1. *Modifying current Executor module*

During this project, we made significant modifications to the executor module of the DISSECT-CF-Fog framework to enhance its efficiency and scalability. Here's a detailed explanation of the steps and challenges we encountered:

Removing Scheduled Annotation:

Initially, the *SimulatorJobExecutorService.java* file used the *@Scheduled* annotation to periodically execute jobs, as shown below:

```
45 |  
46 | // @Scheduled(initialDelay = 0, fixedDelayString = "${dissect-cf.jobs.delay_in_milliseconds}")  
47 | public void runSimulationForTheNextAvailableJob() { no usages  
48 |     final var startTime = System.nanoTime();
```

Code snippet 3.1 - 46th line of the code in SimulatorJobExecutorService.java shows commented line of the Scheduled annotation

This annotation is crucial for triggering job execution at fixed intervals.

Challenges with ProcessBuilder:

After removing the scheduled annotation, our first approach was to use ProcessBuilder to create processes for executing jobs. This approach required running executors through the command line, which presented several challenges [14]:

- **Execution Issues:** Running executors as command-line processes proved to be complex and error-prone.
- **Maven Assembly Plugin:** We attempted to package the application using the Maven Assembly Plugin to create a JAR file. However, this approach did not yield the expected results. The JAR file did not include the necessary executor classes, which are essential for the observer module to detect and manage the executors.

Spring Framework Packaging:

Since our executor is based on the Spring Framework, we decided to package the application as a Spring Boot application using the Maven Spring Boot Plugin. This approach successfully created a runnable JAR file that included all necessary dependencies but without executor module classes. However, we encountered another issue:

- **Artifact Mismatch:** There were inconsistencies in the pom.xml file due to mismatching artifact identifiers (*u-szeged* vs. *u_szeged*). This caused difficulties in resolving dependencies and locating the correct JAR files in the Maven repository.

Resolving Artifact Issues:

To address the artifact mismatch, we meticulously corrected the artifact identifiers in the pom.xml file and ensured that the correct versions of the dependencies were used. This involved:

- **Locating JAR Files:** We manually verified the presence and paths of the required JAR files in the Maven .m2/repository directory.
- **Updating pom.xml:** We updated the pom.xml file to reference the correct artifacts, ensuring that all necessary dependencies were correctly included in the build process.

```
12     <groupId>hu.u-szeged.inf.sed.fog.simulator.executor</groupId>  
13     <artifactId>dissect-cf-fog-executor</artifactId>  
14     <version>1.0.0-SNAPSHOT</version>
```

```
<dependency>  
  <groupId>hu.u-szeged.inf.sed.fog.simulator</groupId>  
  <artifactId>dissect-cf-fog</artifactId>  
  <version>1.0.0-SNAPSHOT</version>  
</dependency>
```

Code snippet 3.1 Differences between executor and simulator modules in pom.xml

Finally, we decided to apply just a removing scheduled annotation and fixing artifact mismatch. The previous packing jar and packing spring application building configurations have been removed from the pom.xml file.

3.2. Improving job-uploading mechanism

Initially, the job-uploading mechanism for the DISSECT-CF-Fog framework was implemented in a single JavaScript file, job-uploader.js, which uploaded one job to the database per call. To enhance performance, several significant improvements were made. Here's an in-depth explanation of the steps we took, and the improvements achieved.

3.2.1 Original Job-Uploader Mechanism

The original script uploaded individual files to MongoDB using GridFS and then created a simulation job with these files. This process was sequential, handling one job at a time, which led to inefficiencies.

- **Job Creation:** After uploading the files, a job entry was created in the *simulator_jobs* collection, referencing the uploaded files by their Object IDs.
- **Manual Configuration:** The Object IDs for the necessary configuration files had to be manually set in the script, which was cumbersome and error prone.

```

25  ✓ async function main() {
26      const database = 'dissect';
27      const uri = `mongodb://localhost:27017/${database}`;
28      const client = new mongodb.MongoClient(uri, { useUnifiedTopology: true });
29
30      try {
31          await client.connect();
32
33          // Upload the necessary XML files
34          const instances = await uploadFile(client, database, './resources/instances.xml', 'instances.xml');
35          const applications = await uploadFile(client, database, './resources/applications.xml', 'applications.xml');
36          const devices = await uploadFile(client, database, './resources/devices.xml', 'devices.xml');
37
38          // Create the simulation job
39          const createdAt = new Date().toISOString();
40          const result = await client.db("dissect").collection("simulator_jobs").insertOne({
41              user: "null",
42              simulatorJobStatus: "SUBMITTED",
43              configFiles: {
44                  APPLIANCES_FILE: new mongodb.ObjectId(applications._id),
45                  DEVICES_FILE: new mongodb.ObjectId(devices._id),
46                  INSTANCES_FILE: new mongodb.ObjectId(instances._id),
47                  IAAS_FILE0: new mongodb.ObjectId("64945d787f86027186921749"),
48                  IAAS_FILE1: new mongodb.ObjectId("64945d787f8602718692174b"),
49              },
50
51              createdAt: createdAt,
52              lastModifiedDate: createdAt,
53          })
54          console.log(result.insertedId);

```

Code snippet 3.2.1 - Previous job-uploader.js file main async method to upload job to the database.

3.2.2 Improvements Implemented to the Job-Uploader.js

To optimize the job-uploading mechanism, the following improvements were made:

- 1) **Batch Processing:** Instead of uploading jobs one by one, the new script processes multiple jobs in a batch. This significantly reduces the time required for job uploads. By grouping multiple jobs together, the script can handle them more efficiently, utilizing the available resources better.

Batch Processing in simple terms:

- The *for* loop iterates 30 times, each iteration representing a new job.
- Creates a unique user identifier for each job.
- Constructs the file paths and initiates asynchronous uploads for the configuration files using *Promise.all()*. This ensures that all file uploads for a job are performed concurrently.

```
const jobs = [];

// Loop to create multiple jobs
for (let i = 1; i <= 30; i++) {
  const user = `user_${i}`;
  const baseFilePath = './resources/';
  const files = ['instances.xml', 'applications.xml', 'devices.xml'].map(file
    uploadFile(client, database, `${baseFilePath}${file}`, `${user}_${file}`
  );
}
```

In the *job-uploader.js* *main()* function creating jobs array and filling it with necessary examples

- 2) **Asynchronous Uploads:** By using asynchronous operations, the system can handle multiple uploads concurrently. This means that the script does not have to wait for one job to finish uploading before starting the next one. Asynchronous uploads improve overall efficiency and reduce the total time taken to upload jobs.

What we did by applying **Asynchronous Uploads:** To be honest asynchronous uploads was implemented in the old version of the file, but we added other methods to improve it

- Waits for all files to upload using *await Promise.all(files)*.
- After the files are uploaded, construct the *configFiles* object with the uploaded file IDs and previously fetched IAAS file IDs.

- 3) **Automatic Object ID Matching:** The improved script automatically fetches and matches the Object IDs for the necessary configuration files from the database. This eliminates the need for manual configuration, ensuring that each job references the

correct configuration files. This is crucial because Object IDs in reset,DB are unique and change every time the database is reset or files are re-uploaded. I faced this problem during the creating observer module, without configuring this we can't run even executor because *SimulatorJobDao* may return null value, which is equal to problem with the file (non-existing file) retrieval.

```
// Fetching ObjectIds for IAAS files => Database from the fs.files
const iaasFile0Id = await getFileObjectId(client, database, "LPDS_16.xml");
const iaasFile1Id = await getFileObjectId(client, database, "LPDS_32.xml");
```

Finding the LPDS_16.xml and LPDS_32.xml files Object_Id

```
const uploadedFiles = await Promise.all(files);
const configFiles = {
  APPLIANCES_FILE: uploadedFiles[1]._id,
  DEVICES_FILE: uploadedFiles[2]._id,
  INSTANCES_FILE: uploadedFiles[0]._id,
  IAAS_FILE0: iaasFile0Id,
  IAAS_FILE1: iaasFile1Id,
};
```

Automatically attaching them to the configuration files sections IAAS_FILE0 and IAAS_FILE1

Overall Achievements and Advantages

- ❖ **Improved Efficiency:** By processing jobs in batches and using asynchronous uploads, the job-uploading mechanism is much more efficient. This reduces the overall time required to upload multiple jobs, making the system more responsive and capable of handling larger workloads.
- ❖ **Reduced Manual Configuration:** Automating the fetching and matching of Object IDs eliminates the need for manual configuration. This reduces the chances of errors and makes the system easier to set up and maintain.

- ❖ **Scalability:** The improvements made to the job-uploading mechanism enhance the scalability of the system. The system can now handle a larger number of jobs simultaneously, making it suitable for more demanding use cases.

3.3. Creating observer module

Abstract

The observer module was designed to extend the features of the executor module by enabling parallel execution of tasks, replacing the dependency on the `@Scheduled` annotation, and introducing enhanced job management capabilities. This section describes the architecture, implementation steps, and key functionality achieved.

A concise overview of the Observer module's purpose, emphasizing its role in enhancing simulation efficiency in IoT-Fog-Cloud environments within the DISSECT-CF-Fog framework. This section will briefly touch on the integration with MongoDB for job management and the replacement of static scheduling with dynamic job execution.

3.3.1 New Architecture and Structure

The Observer module is developed as a Maven project to leverage the dependency management capabilities Maven provides. This is particularly important due to our reliance on MongoDB for job storage and retrieval.

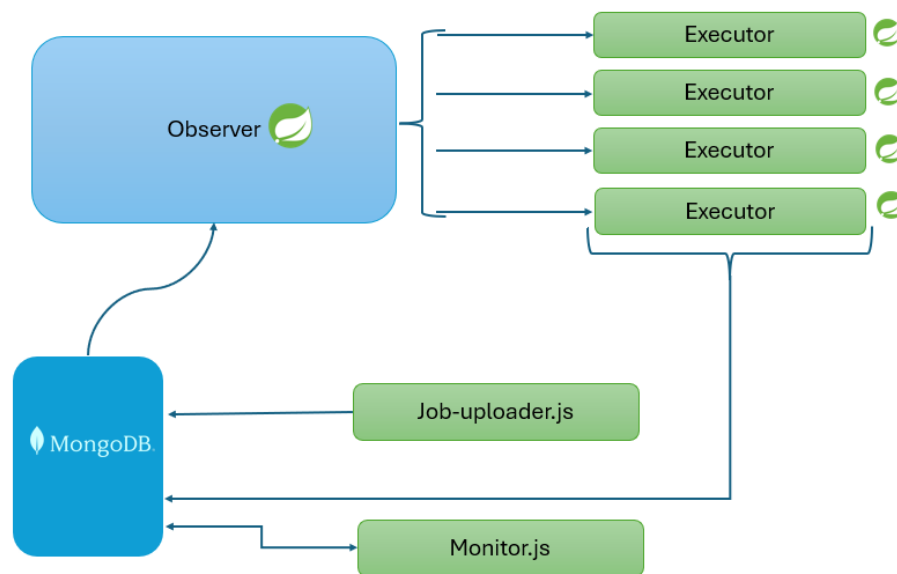


Figure 3.3.1a Planned New Architecture and its connection with other modules and files.

ExecutorService Management

The Observer module utilizes Java's *ExecutorService* for managing concurrency. Specifically, we implemented two *ExecutorService* instances:

Scheduler ExecutorService:

- This executor uses *scheduleAtFixedRate* to periodically check the database for new jobs. This approach ensures that the system is consistently aware of any new jobs that need processing. The Observer periodically queries the MongoDB database for new jobs. This was initially tested and confirmed to be working.

Job Detection and Execution:

- Upon detecting new jobs, the Observer does not process them immediately. Instead, it adds a *Runnable* or *Callable* task to a *newFixedThreadPool(n)*, which manages the execution of these jobs.

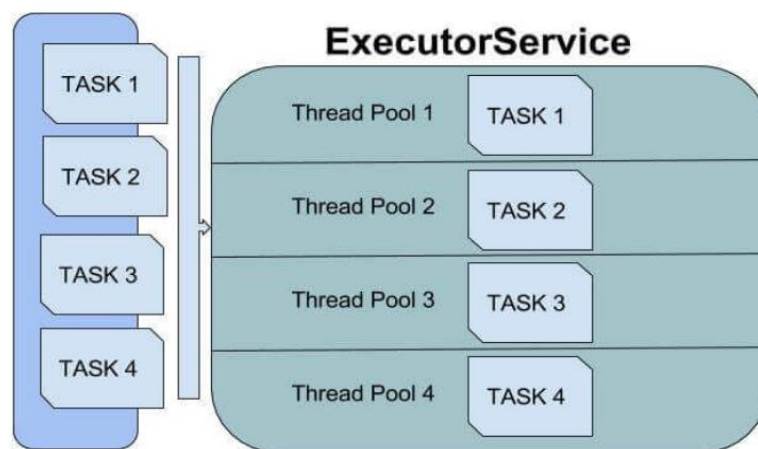


Figure 3.3.1b Java.util.concurrent ExecutorService example of managing tasks

Job Execution ExecutorService:

- This executor handles the actual job execution. When a new job is detected in the database, a *Runnable* or *Callable* task is submitted to a *newFixedThreadPool(n)*, which starts the Spring Boot Executor jar using *ProcessBuilder*.

We had 3 variations of Observer module (2 of them with Processbuilder and one with only JVM Executor module) and finally I chose last variation of the Observer because it showed best performance among planned Observer modules.

So this new project has the same folder structure as other modules. In the pom.xml we added executor module dependency to use executor modules classes in the observer module. It reduces double creation of the same classes or functions which we already have in the Executor module.

To extract and obtain the executor module JAR file, navigate to the root folder of the executor module using the command line: *mvn clean install*.

After successfully generating the dissect-cf-fog-executor.jar file, import it into the observer module. Alternatively, if you are using an IDE like IntelliJ Idea with Maven support, the artifact will be automatically sent to your local Maven repository (*.m2/repository folder*).

3.3.2 Observer components

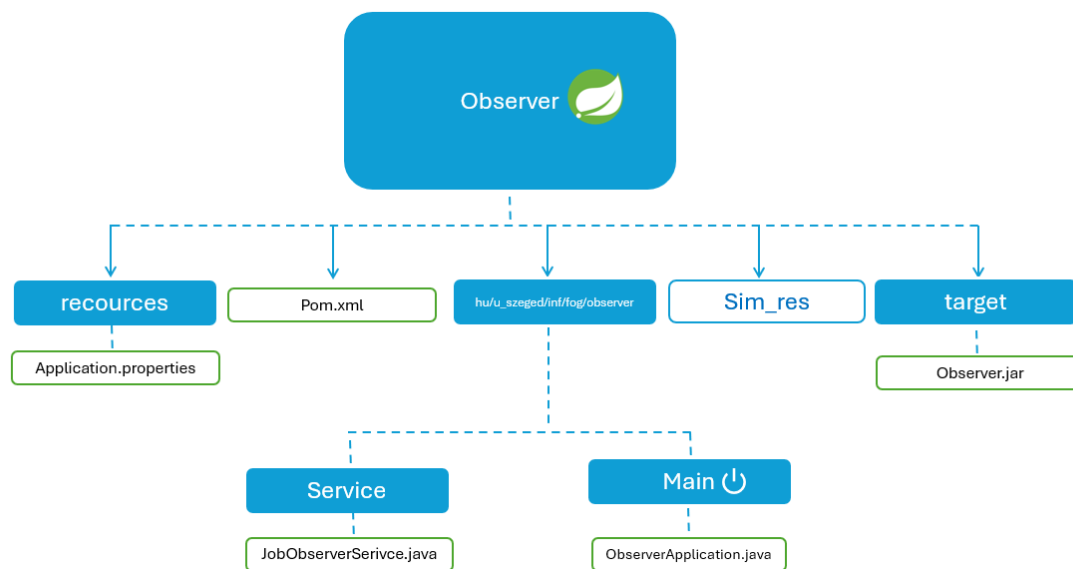


Figure 3.3.2a - Observer module structure and components

The provided diagram illustrates the structure and components of the observer module, which includes the main application entry point, service classes, configuration files, and

resources. This is the top-level module which houses all the components necessary for the observer's functionality.

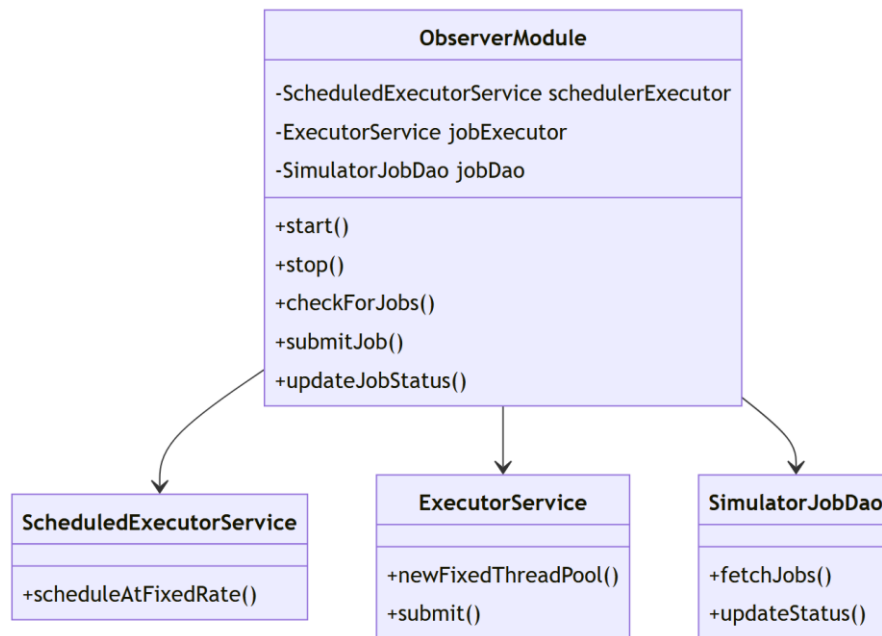


Figure 3.3.2b flowchart of the Observer Module

The class diagram provides a visual representation of the architecture and interactions within the *Observer module*. The *ObserverModule* acts as the central coordinator, leveraging the *ScheduledExecutorService* for periodic job checks, the *ExecutorService* for concurrent job execution, and the *SimulatorJobDao* for database operations.

JobObserverService

This is the core service class of the Observer module. It is responsible for periodically checking the database for new jobs and managing their execution. Key functionalities include:

- **Scheduled Job Checking:** Utilizes a *ScheduledExecutorService* to regularly poll the database for new jobs at a configurable interval.
- **Job Submission:** Submits jobs for execution using an *ExecutorService*, which manages a pool of worker threads for running jobs concurrently.
- **Error Handling:** Implements robust error handling to manage exceptions and maintain system stability.

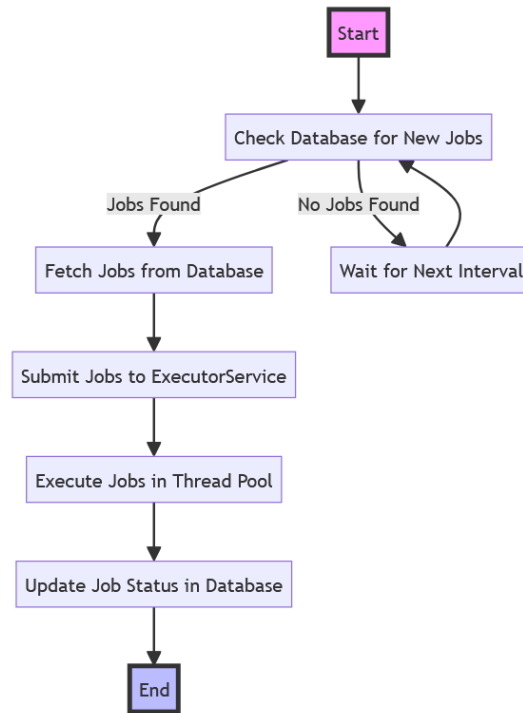


Figure 3.3.2c - Flowchart of the Workflow observer module

The workflow of the Observer module for managing job executions within the DISSECT-CF-Fog framework. Below is a detailed description of each step in the flowchart:

Start is the entry point of the workflow where the Observer module begins its execution process. The Observer module periodically checks the database to see if there are any new jobs that need to be processed. If new jobs are found in the database, the flow proceeds to fetch these jobs. If no jobs are found, the module waits for the next interval to check again. When new jobs are found, they are fetched from the database for processing. The fetched jobs are submitted to the *ExecutorService*, which is responsible for managing their execution using a pool of worker threads. The *ExecutorService* executes the jobs concurrently in a thread pool, ensuring efficient use of resources and faster job completion. After the jobs have been executed, their status is updated in the database to reflect their completion, or any errors encountered during execution. “**END**” marks the end of the workflow for the current cycle. The process will start again at the next scheduled interval.

***checkForJobs()* method:**

This method periodically checks the database for new jobs using the *ScheduledExecutorService*. If new jobs are found, they are fetched from the database.

***submitJob()* method:**

Submits the fetched jobs to the *ExecutorService* for execution. It ensures that jobs are handled efficiently using a thread pool.

***updateJobStatus()* method:**

Updates the status of jobs in the database after they have been executed. This includes marking jobs as completed or handling any errors that occurred during execution.

Implementation Details:

The service uses Spring annotations to manage dependency injection and service lifecycle. It leverages Java's concurrency API to manage task scheduling and execution effectively. Below is a snippet of the service implementation:

```
@Service
public class JobObserverService {
    private ScheduledExecutorService scheduler;
    private ExecutorService jobExecutor;

    @PostConstruct
    public void init() {
        scheduler = Executors.newScheduledThreadPool(jobSchedulerPoolSize);
        jobExecutor = Executors.newFixedThreadPool(jobExecutorThreadPoolSize);
        scheduleJobChecking();
    }

    private void scheduleJobChecking() {
        scheduler.scheduleAtFixedRate(this::checkForJobs, 0, checkInterval, TimeUnit.SECONDS);
    }
}
```

Code snippet 3.3.2 - Initialization of the JobObserverService

3.3.3 Database Integration

The Observer module interacts with a MongoDB database to retrieve and update the status of jobs. It utilizes the *SimulatorJobDao* class to:

- **Fetch Jobs:** Retrieve jobs that are ready for execution.
- **Update Job Status:** Update the job status upon completion or failure.

Implementation Details:

Using Spring Data MongoDB, the module integrates with the database seamlessly. The SimulatorJobDao class abstracts database operations, providing clean and manageable code for database interactions.

Monitoring and Logging

Monitoring and logging are integral to maintaining the reliability of the Observer module. It uses the SLF4J logging framework integrated with Spring Boot's logging capabilities to log application events and errors.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Service
public class JobObserverService {

    private static final Logger logger = LoggerFactory.getLogger(JobObserverService.class); 7 usages
```

Code snippet 3.3.3a - Import statements of the logging part

```
spring.data.mongodb.uri=mongodb://localhost:27017/dissect
job.observer.check.interval=1
job.executor.thread.pool.size=5
job.scheduler.pool.size=1
```

Code snippet 3.3.3b - Application.properties file contains main configurations of the Observer.

The Observer module serves as a dynamic mediator and controller within the DISSECT-CF-Fog framework, facilitating interaction between the Executor module and the system's database, while also interfacing with user commands and system monitoring tools. This section outlines the key interactions and integrations that enable the Observer to perform its functions effectively.

Interaction with the Executor Module

The Observer module is tightly integrated with the Executor module to manage and trigger job executions based on system and user requirements.

- **Job Dispatch:** The Observer retrieves jobs from the database and dispatches them to the Executor for processing. This interaction ensures that jobs are processed as they become available without delay.
- **Execution Feedback:** After job execution, the Executor updates the job's status (e.g., completed, failed) in the database, which the Observer monitors to manage job lifecycle and handle errors or retries.

4. EVALUATION

4.1. Testing with scheduled annotation and with observer module

Objective: To compare the performance of the system when using the traditional scheduled annotation method against the newly implemented observer module. This comparison aims to evaluate efficiency, resource utilization, and job handling capacity. Various configurations were tested, including different thread pool sizes and job upload intervals. The tests were conducted under controlled conditions to ensure reliable and reproducible results.

Methodology:

Scheduled Annotation Tests: Utilized the Scheduled annotation in the Spring Framework to manage job execution at fixed intervals (1 second, 5 seconds, and 10 seconds). Data collected from these tests provided insights into job throughput, execution time, and system stability. Key performance metrics such as job execution time, throughput, and resource utilization were measured and analyzed.

Observer Module Tests: Jobs were managed through the observer module which dynamically checks the job queue and processes jobs without fixed intervals. Various configurations of the observer's parameters (check intervals and thread pool sizes) were tested (e.g., 1 check every 5 seconds with a pool size of 1, 3 checks every 5 seconds with a pool size of 10).

Data Collection:

The monitor-mongo.js script was used to log and extract job processing data directly from MongoDB, detailing job status changes (Submitted, Processing, Processed) over

time. This script creates 2 files jobStatus.xls and jobStatus.log files. Detailed data was collected during the tests, including timestamps, job completion status, and system resource usage.

Job processing data from the observer module under different configurations was collected to measure the effectiveness of parallel processing and system responsiveness.

Analysis Tools:

- Excel was used to visualize job processing times and throughput across different configurations and intervals, allowing for a clear comparative analysis of scheduled versus observer-driven executions.
- In addition, *python [pandas,matplotlib,numpy]* used to visualize the data

4.2. Analyzing differences

Test case	Job Observer Check Interval	Job Executor Thread Pool Size	Job Scheduler Pool Size	Scheduled Fixed Delay
1	10	5	1	10
2	1	5	1	1
3	10	10	3	5

Table 4.2 – Different test cases of the Observer module

Example Test Cases: Various observer parameter configurations (check intervals and thread pool sizes) were tested (for example, 1 check every 5 seconds with a pool size of 1, 3 checks every 5 seconds with a pool size of 10).

How we calculate

Initially, we upload simulation jobs to the database using job-uploader.js within the Node.js environment. Typically, 30 simulations are sufficient for moderate calculations. All jobs are initially assigned a "SUBMITTED" status.

Next, we activate a database scanner to monitor the number of jobs in various states (SUBMITTED, PROCESSING, or PROCESSED) every 5 seconds. After each scan, we save a jobStatus.xls (Excel) file named after the specific configuration being tested.

For instance, if we plan to evaluate each module three times (with and without the Observer, and with scheduled annotation Executor), we'll have six distinct configurations. We must wait for all jobs to be either simulated or reach the "PROCESSED" status.

Occasionally, "PROCESSING" jobs may not transition to "PROCESSED." In such cases, the Observer module includes a shutdown method to gracefully terminate the affected thread.

After completing the six measurements (three with and three without the Observer, with scheduled annotation), we obtain six Excel files for analysis and visualization. These files contain timestamps and information about job statuses (Submitted, Processing, and Processed), with a 5 or 3-second interval between data points (adjustable based on request frequency).

4.2.1 Python script to calculate and visualise the performance

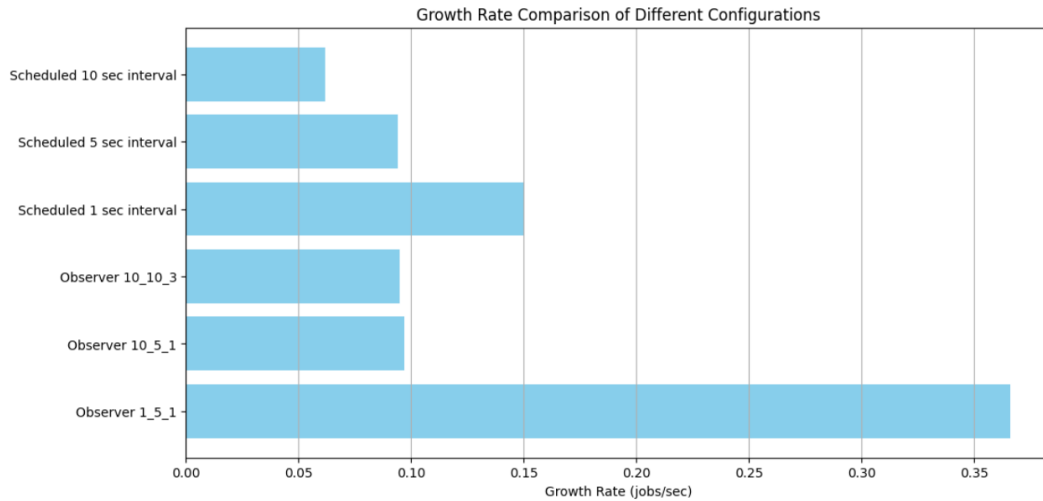
Load the datasets

The datasets for different configurations (Observer and Scheduled) are loaded to analyze their performance in terms of job processing rates. These datasets are Excel files which are measured by running *monitor-mongo.js*.

Calculate Growth Rates

The growth rate for each configuration is calculated using the formula:

$$Growth\ Rate = \frac{Total\ Jobs\ Processed}{Total\ Time}$$

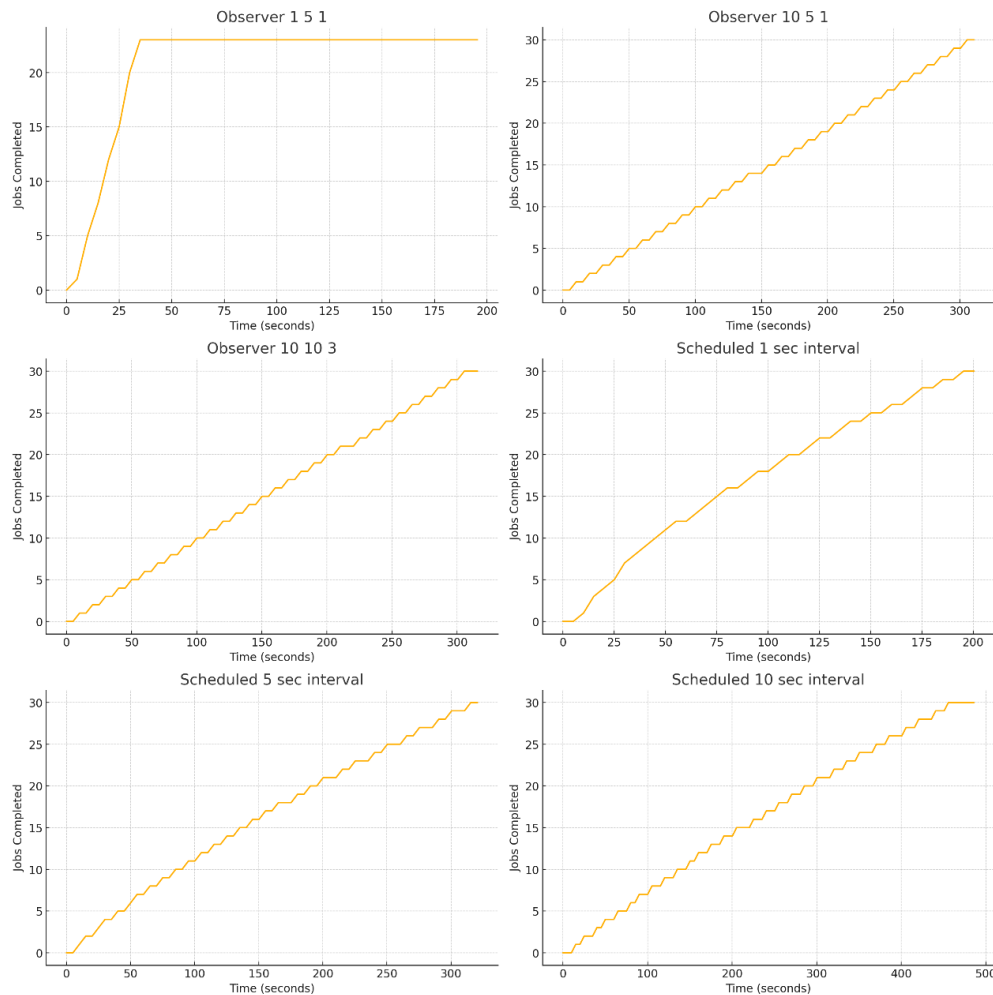


Barchart 4.2.1 – Growth rate Comparison of Different Configurations

Generally, we get better performance in the case of high frequency database checking and when we have a high number of pool sizes. The dynamic and adaptive scheduling mechanism of the Observer module enables it to handle jobs more efficiently, leading to faster processing and better resource utilization.

Typically, we compare the performance of scheduled annotation executor services against observer modules. Recognizing that scheduled executors use a single thread while observers employ multiple threads, a fair comparison involves using the same timer rate for checking jobs in the database.

- **Observer 1.5.1:** Around 23 jobs are completed very quickly, within approximately 25 seconds. This suggests a highly efficient configuration with fast observer implementation and likely a small job queue.
- **Observer 10.5.1:** Job completion takes significantly longer, around 250 seconds. This indicates a slower observer, or a larger job queue compared to Observer 1,5,1.
- **Observer 10.10.3:** Similar to Observer 10,5,1, job completion is gradual and takes about 250 seconds. The difference in parameters (10,10,3 vs. 10,5,1) does not seem to have a major impact on overall completion time.



Linecharts 4.2.1 – based on Excel data. X axis => Time , Y axis => The amount of completed jobs

- **Scheduled 1 sec interval:** Jobs are completed at a steady pace, with a near-linear increase over time. This is expected for a scheduled executor with a 1-second interval, as it processes jobs at a consistent rate.
- **Scheduled 5 sec interval:** Similar to the 1-second interval, but with a slower completion rate due to the longer interval between job executions.
- **Scheduled 10 sec interval:** The slowest completion rate among the scheduled executors, with a roughly linear increase in completed jobs over time. This is consistent with the 10-second interval between job executions.

Growth Rates of Completed Jobs: Based on python script calculation.

Fastest Growing: Observer 1,5,3 with a growth rate of 0.366 jobs per second.

Slowest Growing: Scheduled 10 sec interval with a growth rate of 0.062 jobs per second.

Configuration	Growth Rate (jobs/sec)
Observer 1_5_1	0.366
Observer 10_5_1	0.097
Observer 10_10_3	0.095
Scheduled 1 sec interval	0.150
Scheduled 5 sec interval	0.094
Scheduled 10 sec interval	0.062

Table 4.2.1 – Growth rate of different simulations

The data extracted from the tests revealed specific cases where the observer module significantly outperformed the scheduled tasks, particularly in scenarios of high job density and varied execution times.

1. Efficiency in Job Processing:

- The Observer module (particularly the 1,5,1 configuration) significantly outperformed the scheduled annotation method. It demonstrated a growth rate of 0.366 jobs/sec, which is approximately 6 times faster than the slowest growing configuration (Scheduled 10 sec interval).
- The adaptive scheduling mechanism of the Observer module allowed it to dynamically allocate resources and execute jobs more efficiently, leading to quicker job completion times and better overall system performance.

2. System Throughput:

- The throughput, measured as the number of jobs processed per second, was highest for the Observer 1,5,1 configuration. This indicates that the Observer module's ability to dynamically check for and execute jobs without fixed intervals provides a significant advantage in handling high job densities and varied execution times.

3. Resource Utilization:

- The Observer module demonstrated improved resource utilization by leveraging real-time data to adaptively schedule tasks. This approach reduced idle time and ensured that computational resources were utilized more effectively.

- The scheduled annotation method, while simpler to implement, led to periods of underutilization or overloading, depending on the fixed intervals set for job execution.
4. **Scalability and Adaptability:**
- The Observer module's architecture, which uses Java's `ExecutorService` for managing concurrency, proved to be highly scalable. It handled increasing job loads efficiently by dynamically adjusting to the workload.
 - The modular design of the Observer module allows for easy extensibility and integration of new features, making it a robust solution for evolving IoT-Fog-Cloud environments.

5. Conclusion

5.1. Overview

The modifications to the executor module, improvements in the job-uploading mechanism, and the development of the Observer module have significantly enhanced the performance and scalability of the DISSECT-CF-Fog framework. The results of the evaluation demonstrate clear improvements in job execution time, throughput, and resource utilization.

The development and integration of the Observer module within the DISSECT-CF-Fog framework have led to significant improvements in job processing efficiency, system throughput, and resource utilization.

The Observer module's dynamic and adaptive scheduling mechanism allows it to outperform traditional scheduled methods, making it a highly effective solution for managing tasks in complex IoT-Fog-Cloud environments.

5.2. Ideas for the further Development

- 1) **Machine Learning Integration:** Integrating machine learning algorithms to predict job execution times and optimize resource allocation dynamically.
- 2) **Advanced Monitoring:** Enhancing the Observer module with more advanced monitoring and alerting capabilities to proactively address potential issues.
- 3) **Scalability Enhancements:** Further optimizing the system for large-scale deployments, including distributed execution and multi-node support.
- 4) **User Interface Improvements:** Developing a more user-friendly interface for configuring and managing simulations, providing better visibility into system performance and job status.

Bibliography

- [1]. Armbrust, M., Fox, A., Griffith, R., Joseph, A. D., Katz, R., Konwinski, A., ... & Zaharia, M. (2010). A view of cloud computing. *Communications of the ACM*, 53(4), 50-58.
- [2]. Mell, P., & Grance, T. (2011). The NIST definition of cloud computing. NIST Special Publication, 800(145), 7.
- [3]. Bonomi, F., Milito, R., Zhu, J., & Addepalli, S. (2012, August). Fog computing and its role in the internet of things. In *Proceedings of the first edition of the MCC workshop on Mobile cloud computing* (pp. 13-16).
- [4]. Yi, S., Li, C., & Li, Q. (2015, June). A survey of fog computing: Concepts, applications and issues. In *Proceedings of the 2015 Workshop on Mobile Big Data* (pp. 37-42).
- [5]. Atzori, L., Iera, A., & Morabito, G. (2010). The internet of things: A survey. *Computer networks*, 54(15), 2787-2805.
- [6]. Gubbi, J., Buyya, R., Marusic, S., & Palaniswami, M. (2013). Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29(7), 1645-1660
- [7]. Gupta, H., Dastjerdi, A. V., Ghosh, S. K., & Buyya, R. (2017). iFogSim: A toolkit for modeling and simulation of resource management techniques in the Internet of Things, edge and fog computing environments. *Software: Practice and Experience*, 47(9), 1275-1296.
- [8]. Calheiros, R. N., Ranjan, R., De Rose, C. A., & Buyya, R. (2009). CloudSim: A novel framework for modeling and simulation of cloud computing infrastructures and services. *arXiv preprint arXiv:0903.2525*.
- [9]. Stojmenovic, I., & Wen, S. (2014). The fog computing paradigm: Scenarios and security issues. In *2014 federated conference on computer science and information systems* (pp. 1-8). IEEE.
- [10]. Yi, S., Hao, Z., Qin, C., & Li, Q. (2015). Fog computing: Platform and applications. In *2015 Third IEEE Workshop on Hot Topics in Web Systems and Technologies (HotWeb)* (pp. 73-78). IEEE
- [11]. [DISSEC_CF-Fog Github Repository](#) (visited: 12.02.2024)
- [12]. [Java Executor Service](#) (visited: 15.02.2024)
- [13]. [Location-aware Task Allocation Strategies for IoT-Fog-Cloud Environments](#) (visited: 25.03.2024) [April 2021, 29th Euromicro International Conference on Parallel, Distributed and Network-based Processing (PDP 2021)]

Declaration

I, Akbar Makhmanzarov, declare that my dissertation was made at Faculty of Science and Informatics, University of Szeged, to obtain a master's degree in computer science.

I declare that I have not presented this thesis for other degrees before, and that I used only my own work, and the sources mentioned (publications, tools...). I acknowledge that my thesis is stored in the Diploma Thesis Repository of the University of Szeged.

Date: 22/05/2024

A handwritten signature in blue ink, appearing to be 'Akbar Makhmanzarov', with a stylized flourish at the end.

Acknowledgements

I would like to express my gratitude to my supervisor, **Andras Markus**, for his invaluable guidance and support throughout this project. He supported me a lot, even when he was in the abroad, he keep the contact and helped me to route correct direction. I also thank the University of Szeged for providing the necessary resources and facilities. Special thanks to my family and friends for their constant encouragement and support.